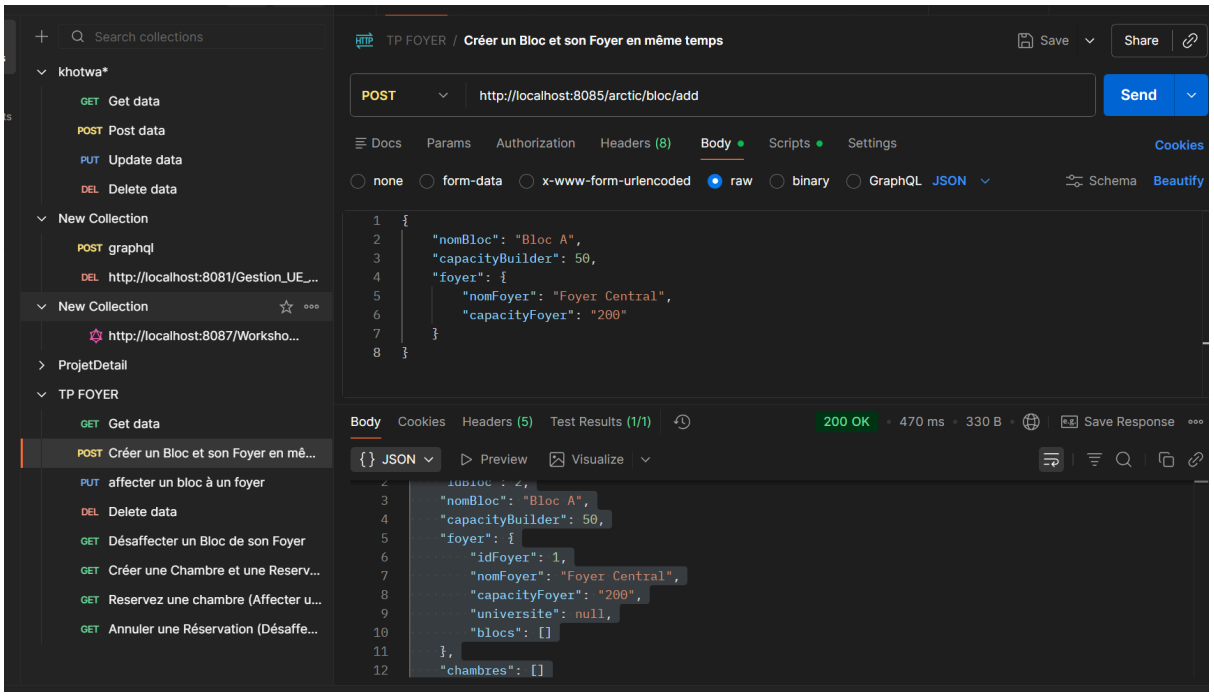
TP Affectation

Exo 1 :

- Créer un Bloc et son Foyer en même temps.

```
@ManyToOne(cascade = CascadeType.ALL)
Foyer foyer;
```



- Affecter un Bloc à un Foyer (Les deux sont déjà dans la base de données).

dans l'interface `IBlocService`

```
Bloc desaffecterBlocDeFoyer(long idBloc);
```

et puis dans le service :

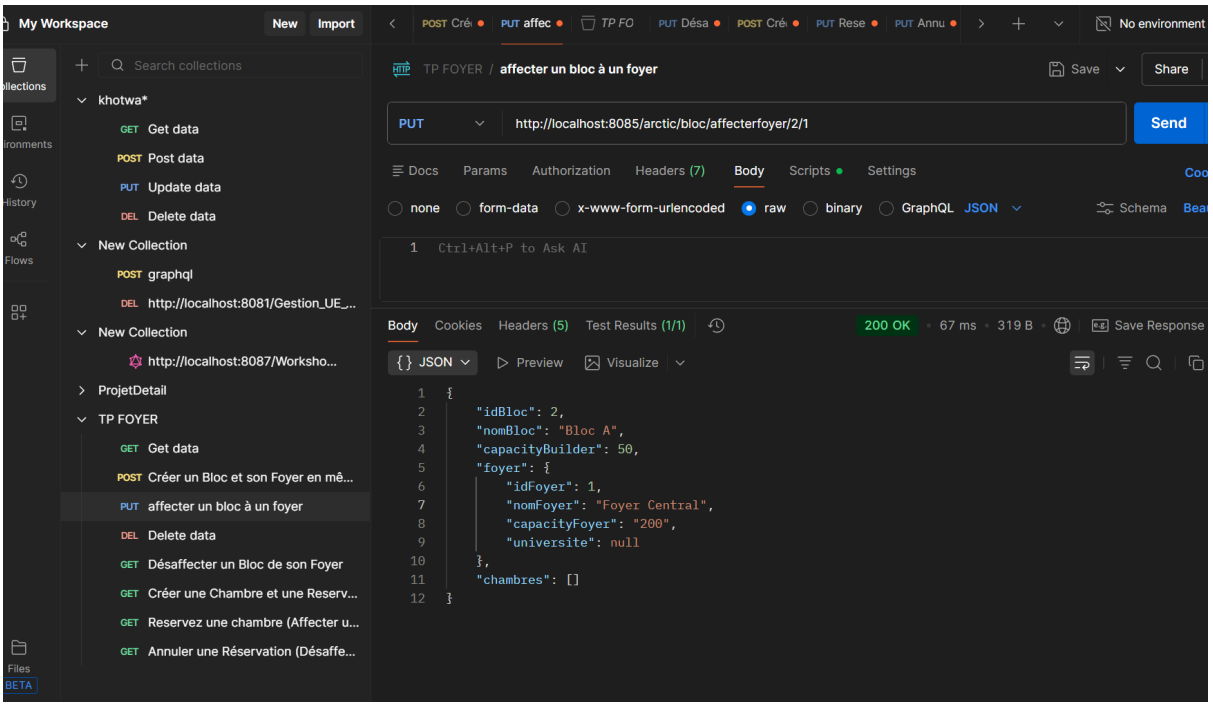
```
@Override
public Bloc affecterBlocAFoyer(long idBloc, long idFoyer) {
```

```
    Bloc bloc = blocRepository.findById(idBloc).get();


    Foyer foyer = foyerRepository.findById(idFoyer).get();


    bloc.setFoyer(foyer);


    return blocRepository.save(bloc);
}
```

et aprés dans le controller :

```
@PutMapping("/affecterfoyer/{idBloc}/{idFoyer}")
public Bloc affecterBlocAFoyer(@PathVariable long idBloc,
@PathVariable long idFoyer) {
    return blocService.affecterBlocAFoyer(idBloc, idFoyer);
}
```

RESULTAT :



- Désaffecter un Bloc de son Foyer

dans l'interface :

```
Bloc desaffecterBlocDeFoyer(long idBloc);
```

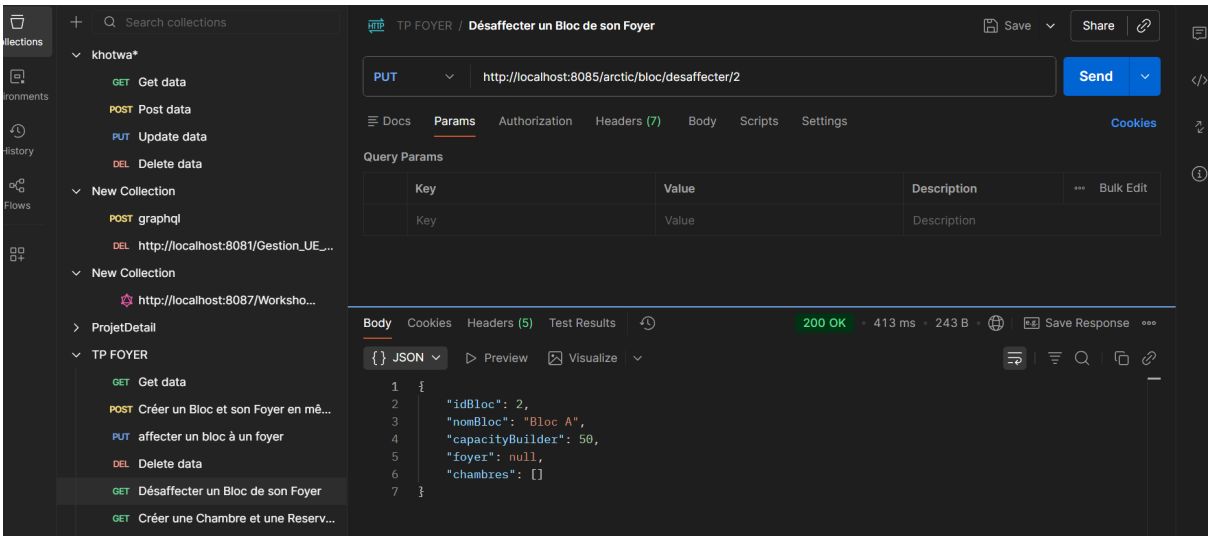
dans le service :

```
@Override
public Bloc desaffecterBlocDeFoyer(long idBloc) {
    Bloc bloc = blocRepository.findById(idBloc).orElse(null);

    if (bloc != null) {
        bloc.setFoyer(null);
        return blocRepository.save(bloc);
    }
    return null;
}
```

dans le controller :

```
@PutMapping("/desaffecter/{idBloc}")
public Bloc desaffecterBlocDeFoyer(@PathVariable long idBloc) {
    return blocService.desaffecterBlocDeFoyer(idBloc);
}
```

RESULTAT

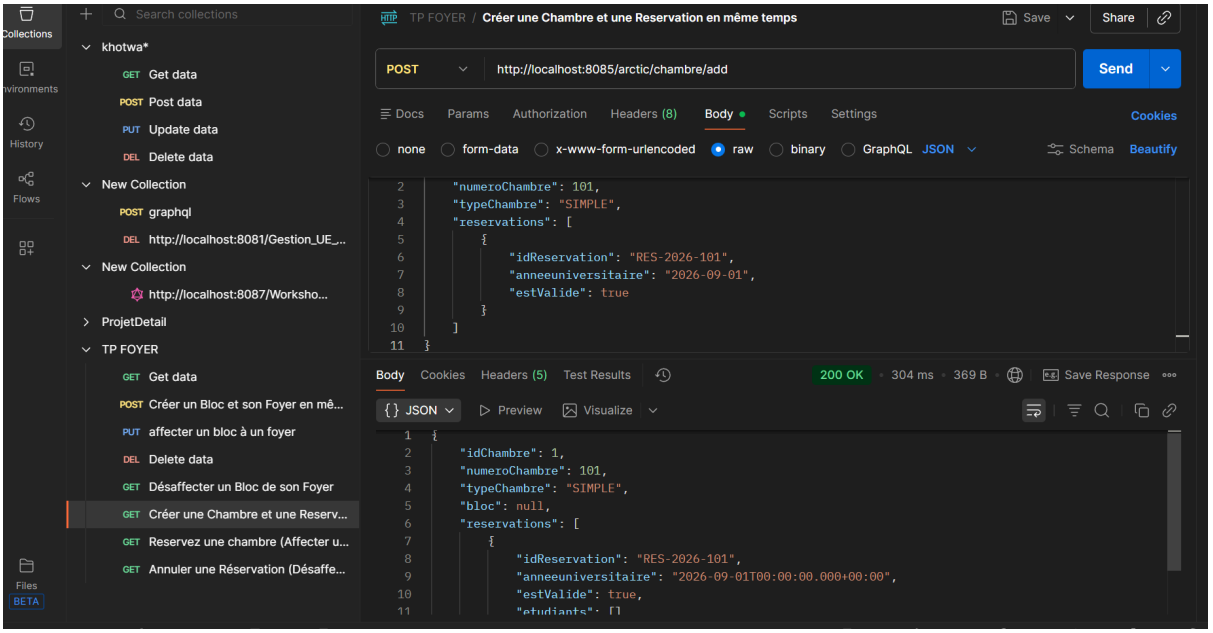


Exo 2 :

- Créer une Chambre et une Reservation en même temps.

```
@OneToMany (cascade = CascadeType.ALL)
Set<Reservation> reservations = new HashSet<>();
```

RESULTAT



- Reservez une chambre (Affecter une Reservation à une Chambre)

dans l'interface :

```
Chambre affecterReservationAChambre(String idReservation, long
idChambre);
```

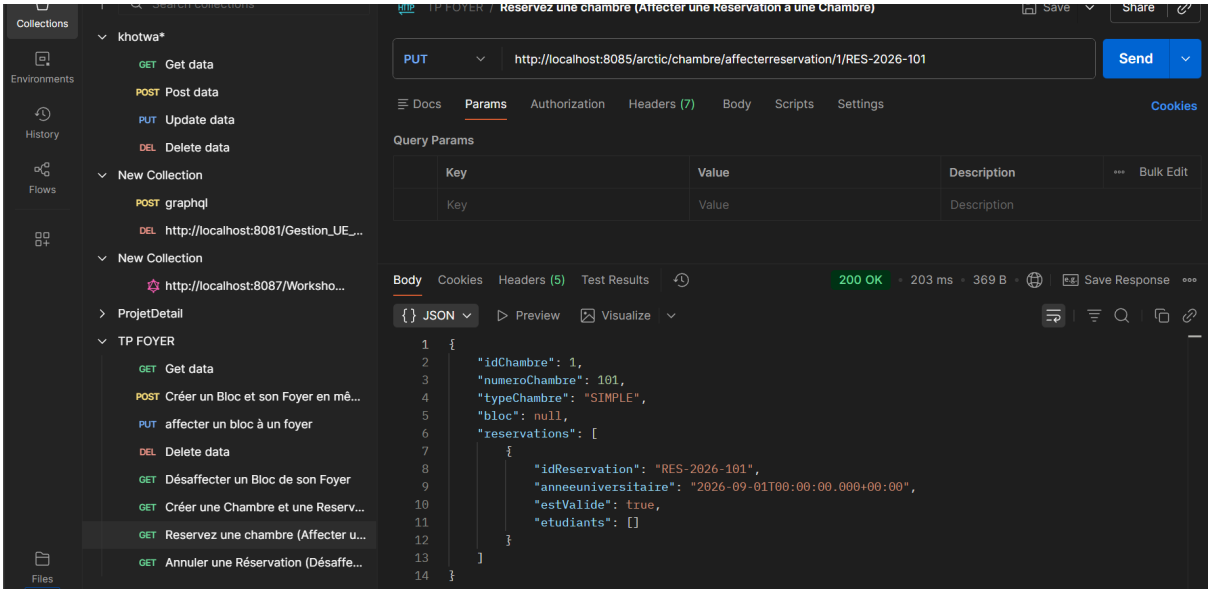
dans le service :

```
@Override
public Chambre affecterReservationAChambre(String idReservation,
long idChambre) {
    Chambre chambre = chambreRepository.findById(idChambre).get();

    Reservation reservation =
reservationRepository.findById(idReservation).get();

    chambre.getReservations().add(reservation);
    return chambreRepository.save(chambre);
}
```

dans le controller

```
@PutMapping("/affecterreservation/{idChambre}/{idReservation}")
public Chambre affecterReservationAChambre(@PathVariable String
idReservation,@PathVariable long idChambre) {
    return
chmabreService.affecterReservationAChambre(idReservation,
idChambre);
}
```

RESULTAT



- Annuler une Réservation (Désaffecter une Reservation d'une Chambre)

dans l'interface :

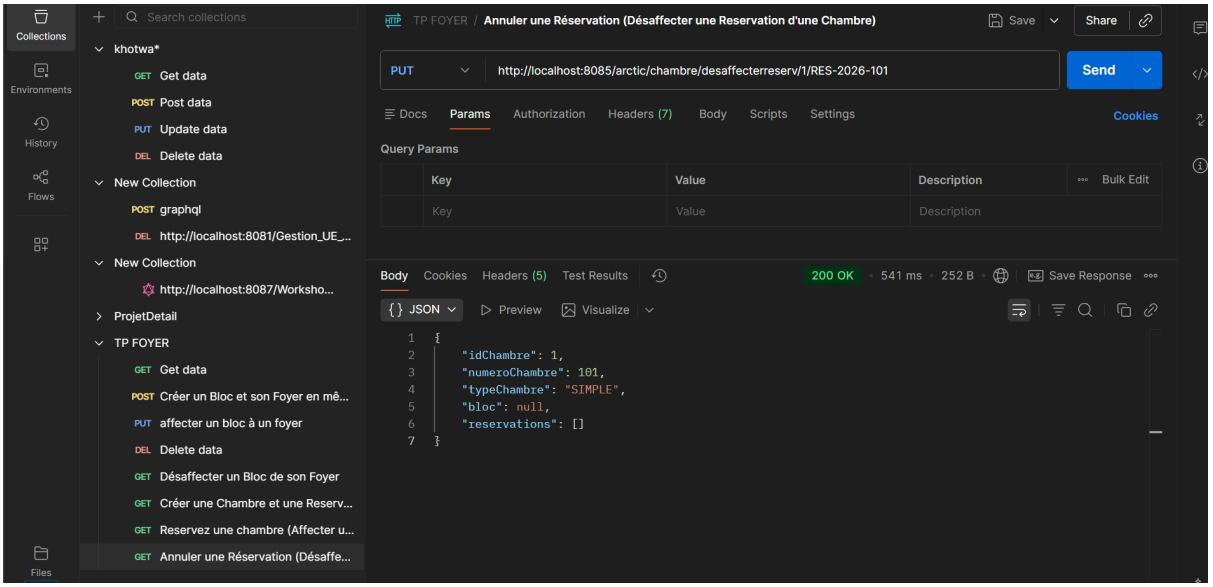
```
Chambre desaffecterReservationDeChambre(String idReservation, long
idChambre);
```

dans le service:

```
@Override
public Chambre desaffecterReservationDeChambre(String
idReservation, long idChambre) {
   Chambre chambre =
chambreRepository.findById(idChambre).orElse(null);

   Reservation reservation =
reservationRepository.findById(idReservation).orElse(null);

   if (chambre != null && reservation != null) {
       chambre.getReservations().remove(reservation);

       return chambreRepository.save(chambre);
   }
   return null;
```

dans le controller :

```
@PutMapping("/desaffecterreserv/{idChambre}/{idReservation}")
public Chambre desaffecterReservationDeChambre(@PathVariable long
idChambre,@PathVariable String idReservation) {
```

```
    return
chmabreService.desaffecterReservationDeChambre(idReservation,
idChambre);
}
```

RESULTAT: